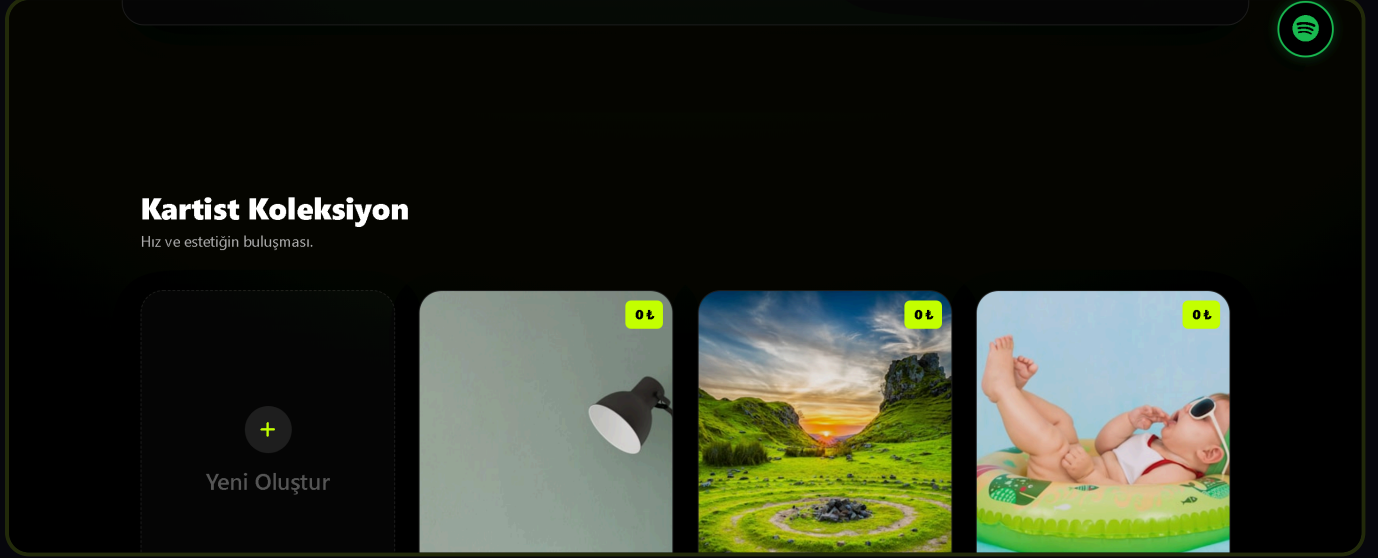


KARTIST v3.0

Yapay Zekâ Destekli SaaS Tasarım Platformu & Tasarımcı Sosyal Ağı

Dönem Geneli (Hafta 1-12 / Sprint 1-5) Geliştirme Süreci ve Teknik Dokümantasyon



ÜNİVERSİTE / BÖLÜM

Sinop Üni. / İnternet ve Ağ Tek.

GELİŞTİRİCİ (ÖĞRENCİ)

Keremcan ATAŞ (247017033)

CANLI DEMO

kartistt.com.tr

PROJE DANIŞMANI

Öğr. Gör. Ekrem SAYDAM

PROJE DURUMU

Sprint 1-5 Tamamlandı ✓

KAYNAK KOD

Keremcanatass-58/Kartist-Editor

Final Rapor Özeti: Vize tesliminde MVP (Sprint 1–3: güvenlik, kimlik doğrulama, AI editör) tamamlanmıştı. Final döneminde proje, salt bir tasarım editöründen **tam teşekküllü bir tasarımcı sosyal ağına** dönüştü: sosyal akış, anlık mesajlaşma, **WebRTC tabanlı canlı yayın**, oyunlaştırma (XP/seviye/rozet) ve **AI jüri** tasarım yarışmaları eklendi. Tüm sistem **.NET 8 + Dapper + SQL Server** üzerinde, **kartistt.com.tr** adresinde canlı çalışmaktadır.

5

TAMAMLANAN SPRINT

12

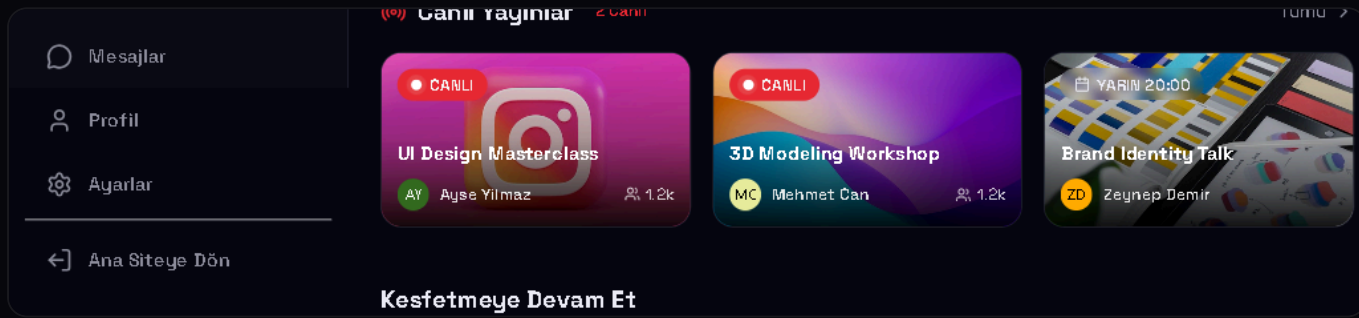
GELİŞTİRME HAFTASI

21+

VERİTABANI TABLOSU

9+

GÜVENLİK KATMANI



Şekil 1 — Kartist Sosyal Akış (Feed): gönderiler, beğeni, yorum, takip ve canlı bildirimler

🎯 Vizeden Finale: Ne Değişti?

Vize sürümü (v2.0) bir **bireysel tasarım aracıydı**. Final sürümü (v3.0) ise bir **topluluk platformudur**. Kullanıcılar artık tasarımlarını paylaşır, birbirini takip eder, gerçek zamanlı yayın açar, düello ve yarışmalara katılır; sistem onları XP ve rozetlerle ödüllendirir. Bu dönüşüm, dersin odağı olan **internet ve ağ teknolojilerini** (HTTP REST, WebSocket/SignalR, WebRTC P2P, STUN/ICE, TLS, OAuth) doğrudan uygulamaya geçirme fırsatı sundu.

ASP.NET CORE 8.0

DAPPER

SIGNALR

WEBRTC

GEMINI / GROQ AI

BCRYPT + 2FA

HMAC CI/CD

Proje, 4 Mart – 29 Mayıs 2026 tarihleri arasında **haftalık iteratif (Agile/Sprint) bir yöntemle** geliştirildi. Her hafta net bir hedef etrafında planlandı, kodlandı, test edildi ve **GitHub Actions / HMAC imzalı deploy** ile canlıya alındı. Aşağıdaki çizelge tüm dönemi özetler; ilerleyen sayfalarda her hafta ayrıntılandırılmıştır.

Hafta 1 — Proje Kurulumu & İskelet (3–9 Mart)

ASP.NET Core MVC projesi, Dapper + SQL Server bağlantısı, ana sayfa (landing), temel yönlendirme ve katmanlı mimarinin oluşturulması.

Hafta 2 — Sprint 1: Güvenlik Altyapısı (10–16 Mart)

SecurityHeaders middleware (CSP), BCrypt şifreleme, global CSRF, InputValidator, parametrik Dapper sorguları, rate limiting.

Hafta 3 — Sprint 2: Kimlik Doğrulama & CI/CD (17–23 Mart)

Cookie Authentication, Google OAuth 2.0, e-posta OTP ile 2FA, hesap kilitleme, dashboard yeniden tasarımı, GitHub Actions pipeline.

Hafta 4 — Sprint 3: AI & Canvas — Vize Teslimi (24–30 Mart)

Gemini/Groq LLM, AI görsel üretimi + fallback zinciri, Fabric.js tasarım tuvali, JSON state kayıt. **29 Mart: Vize raporu teslim edildi.**

Hafta 5–6 — Sprint 4: Sosyal Ağ (31 Mart–13 Nisan)

Gönderi akışı (CRUD), beğeni, yorum (iç içe yanıt), takip sistemi, hikâyeler, AI içerik moderasyonu, keşfet sayfası.

Hafta 7 — Mesajlaşma & Keşif (14–20 Nisan)

Direkt mesaj (DM) sistemi, Instagram/WhatsApp tarzı arayüz yenileme, site geneli kullanıcı keşfi.

Hafta 8 — Sprint 5: Güvenlik Sertleştirme (21–27 Nisan)

Hardcoded secret kaldırma → HMAC imzalı deploy, admin yetki + CSRF, çerez sertleştirme, plaintext şifre kaldırma, CSPRNG 2FA, magic-byte dosya doğrulama.

Hafta 9 — Profil, Liderlik & Oyunlaştırma (28 Nisan–4 Mayıs)

Profil/keşfet/liderlik/pano sayfaları, XP & seviye sistemi, rozetler, günlük görevler, istatistik panosu.

Hafta 10 — Canlı Yayın (WebRTC) & Düello (5–11 Mayıs)

SignalR sinyalizasyon + WebRTC P2P canlı yayın, izleyici sayacı, canlı sohbet, tasarım düelloları arayüzü.

Hafta 11 — Üretim Kararlılığı & Deploy (12–18 Mayıs)

SMTP/iletişim sertleştirme, deploy boru hattı iyileştirmeleri, AI yazar sağlayıcı fallback'i.

Hafta 12 — Yarışma Sistemi & Final Cila (19–29 Mayıs)

AI jüri tasarımları yarışmaları (yaşam döngüsü servisi), 3 varyasyonlu AI Studio, kurumsal canlı harita, canlı yayın kasma/yankı optimizasyonu.

**Backend**

ASP.NET Core 8.0
MVC + Razor Views
Minimal API (deploy)

**Veritabanı**

Microsoft SQL Server
Dapper Micro-ORM
21+ tablo, parametrik

**Gerçek Zamanlı**

SignalR (WebSocket)
WebRTC P2P
STUN / ICE NAT geçişi

**Yapay Zekâ**

Gemini 2.0 / Groq LLM
OpenAI + Pollinations
AI moderasyon & jüri

**Güvenlik**

BCrypt + 2FA (OTP)
CSP, CSRF, Rate Limit
HMAC-SHA256 deploy

**DevOps**

GitHub Actions CI
IIS / Plesk (Windows)
app_offline.htm deploy

Katmanlı Mimari & İstek Boru Hattı

// Kartist v3.0 – İstek Boru Hattı (Program.cs)

İSTEMCİ Browser • Fabric.js • SignalR • WebRTC

MIDDLEWARE SecurityHeaders(CSP) → RateLimit →
Auth Cookie → AntiForgery(global)

CONTROLLER Home • Account • Admin • Social
(BaseController → CurrentUserId)

SERVİS AiPrompt • AiImage • AiModeration •
Social • AiJury • YarismaLifecycle

VERİ ERIŞİMİ Dapper + MSSQL (parametrik @param)

GERÇEK ZAMANLI HUB NotificationHub (SignalR)
/notificationHub • WebRTC sinyalizasyon

Mimari Kararlar:

- **Dapper (EF değil):** Saf SQL kontrolü + parametrik sorgu ile SQL Injection güvencesi.
- **SignalR:** Bildirim ve canlı sohbet için kalıcı WebSocket bağlantısı.

- **WebRTC (mesh P2P):** Medya sunucusu olmadan, tarayıcıdan tarayıcıya düşük gecikmeli yayın.
- **BaseController:** Oturum kullanıcı kimliği tek noktada (CurrentUserId) çözülür.

HAFTA 1 3-9 Mart Proje iskeleti, MVC + Dapper + SQL Server, ana sayfa

HAFTA 2 10-16 Mart Sprint 1: Siber güvenlik altyapısı (OWASP Top 10)



Güvenlik Temelleri

✓ TAMAMLANDI

- ✓ **İSKELET** ASP.NET Core MVC projesi kuruldu; Dapper ile SQL Server bağlantısı ve katmanlı klasör yapısı (Controllers / Services / Data / Helpers) oluşturuldu.
- ✓ **SECURITY HEADERS** X-Frame-Options, X-Content-Type-Options, Referrer-Policy ve **Content-Security-Policy** her yanıtta özel middleware ile eklendi.
- ✓ **BCRYPT** Şifreler BCrypt.Net adaptif hash ile saklanır; plaintext tamamen kaldırıldı.
- ✓ **CSRF (GLOBAL)** AutoValidateAntiforgeryToken global filtre olarak eklendi — tüm POST'lar token zorunlu.
- ✓ **SQL INJECTION ÖNLEME** Tüm sorgular @param ile parametrik; string birleştirme yasak.

</> SecurityHeadersMiddleware.cs

```
// Her HTTP yanıtına güvenlik başlıkları ekleyen middleware
public async Task InvokeAsync(HttpContext context) {
    context.Response.Headers.Append("X-Frame-Options", "SAMEORIGIN");
    context.Response.Headers.Append("X-Content-Type-Options", "nosniff");
    context.Response.Headers.Append("Referrer-Policy",
        "strict-origin-when-cross-origin");

    // Content-Security-Policy: kaynak (script/style/connect) kısıtları
    context.Response.Headers.Append("Content-Security-Policy", csp);
    context.Response.Headers.Remove("Server"); // parmak izi gizle
    await _next(context);
}
```

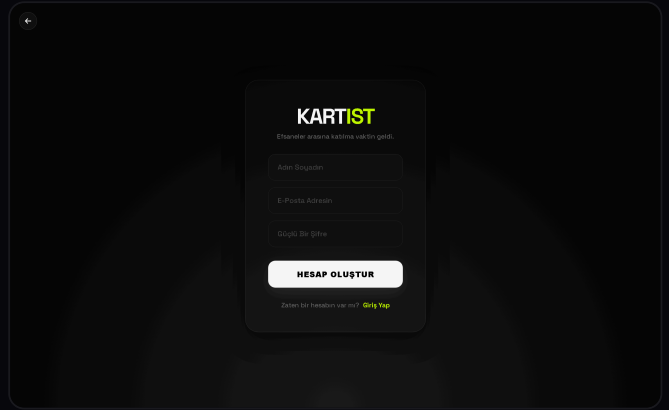
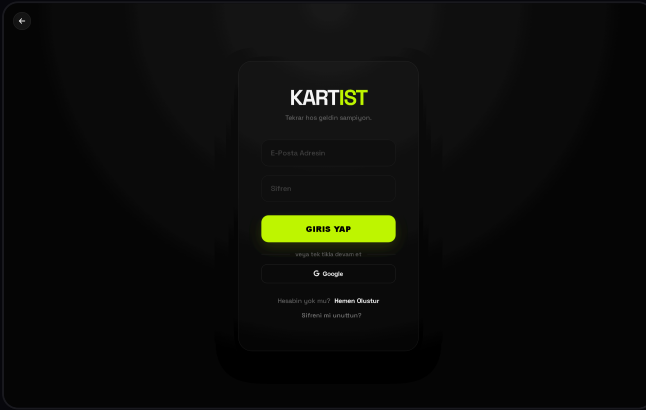
ⓘ Ağ/Güvenlik Notu: CSP, tarayıcıya hangi kaynaktan script/stil/bağlantı yükleyebileceğini bildiren bir HTTP yanıt başlığıdır ve XSS saldırılarını sınırlar. Statik yollar (/lib/, /css/, /uploads/) ve OAuth callback'leri başlıktan muaf tutulur.

HAFTA 3 17–23 Mart Cookie Auth, Google OAuth, 2FA, hesap kilidi, GitHub Actions

Gelişmiş Kimlik Yönetimi

✓ TAMAMLANDI

- ✓ **COOKIE AUTH** KartistCookie — 30 günlük, HttpOnly + Secure + SameSite=Lax oturum çerezi.
- ✓ **GOOGLE OAUTH 2.0** Tek tıkla giriş; yalnızca ClientId+Secret tanımlıysa kaydedilen koşullu OAuth akışı.
- ✓ **2FA (E-POSTA OTP)** Şifre sonrası 6 haneli, 5 dakika geçerli, kriptografik RNG ile üretilen tek kullanımlık kod.
- ✓ **HESAP KORUMASI** Hesap sayımı (enumeration) sızıntısını önleyen sabit-yanıt "şifremi unuttum" akışı.
- ✓ **GITHUB ACTIONS** Her push'ta otomatik restore + build çalıştıran CI pipeline.



Şekil 2 — Giriş (Google OAuth + e-posta/şifre) ve Kayıt ekranları

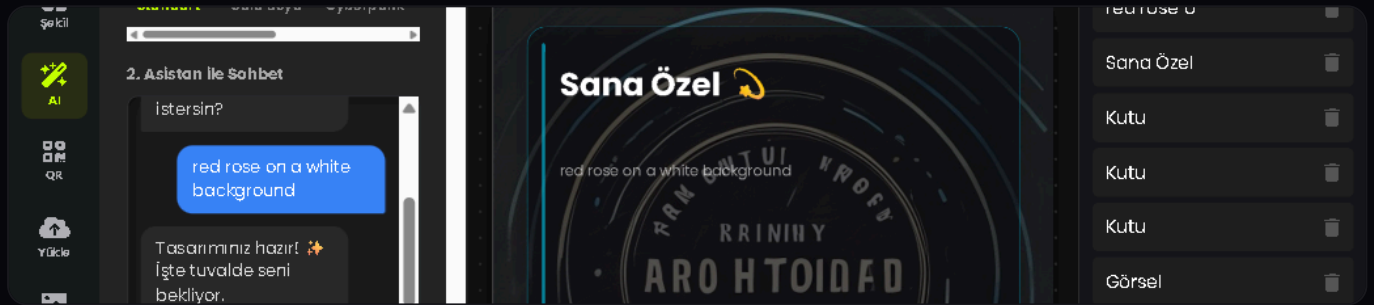
```
// 2FA kodu – kriptografik güvenli RNG (System.Random DEĞİL)
int kod = RandomNumberGenerator.GetInt32(100000, 1000000);
db.Execute(@"INSERT INTO İkiFactorKodlari
  (KullaniciEmail, Kod, BitisTarihi)
VALUES (@mail, @kod, @bitis)",
  new { mail, kod, bitis = DateTime.UtcNow.AddMinutes(5) });
```

HAFTA 4 24–30 Mart Gemini/Groq LLM, AI görsel + fallback, Fabric.js tuval — 29 Mart Vize

Yapay Zekâ Motoru & Tasarım Tuvali

✓ TAMAMLANDI (MVP)

- ✓ **PROMPT (LLM)** Gemini 2.0 Flash (varsayılan) / Groq / OpenAI sağlayıcı zinciri ile kart içeriği, başlık ve renk önerisi üretimi.
- ✓ **GÖRSEL ÜRETİMİ** OpenAI gpt-image-1 → Pollinations → anahtar-kelime stok fallback'i; UI asla hata vermez.
- ✓ **FABRIC.JS TUVAL** Sürükle-bırak yazı/şekil/görsel, katman yönetimi, döndürme/boyutlandırma.
- ✓ **JSON STATE** Tasarımın tüm durumu JSON'a serileştirilip SQL Server'a kaydedilir; kaldığı yerden devam.



Şekil 3 — AI Studio: doğal dil komutundan tasarım üretim akışı

```
// AI görsel üretimi — çok katmanlı fallback (UI hard-fail etmez)
var result = await TryOpenAiAsync(prompt, ct);
if (result.Success) return result;
var poll = await TryPollinationsAsync(prompt, ct); // Katman 2
if (poll.Success) return poll;
return KeywordStockFallback(prompt); // Katman 3
```

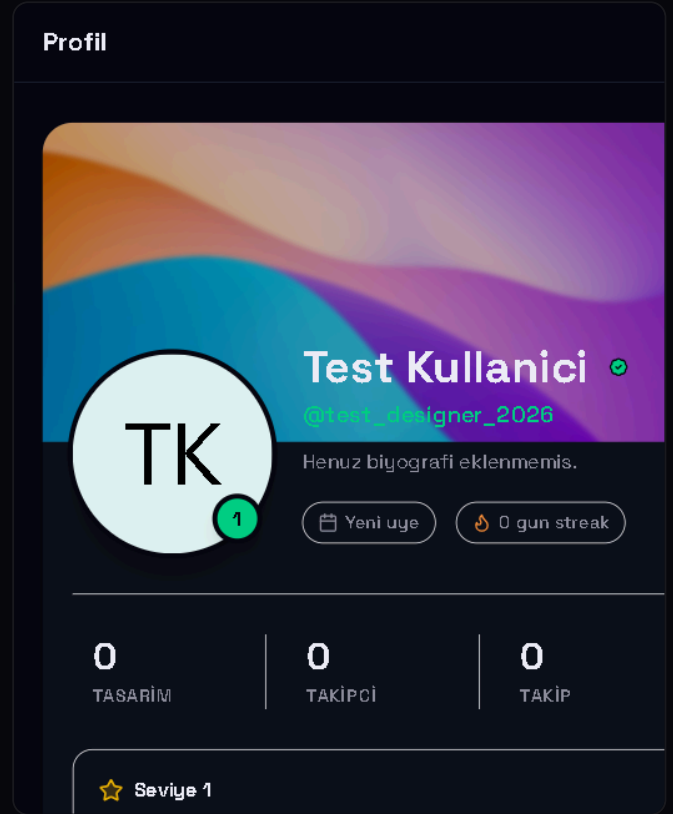
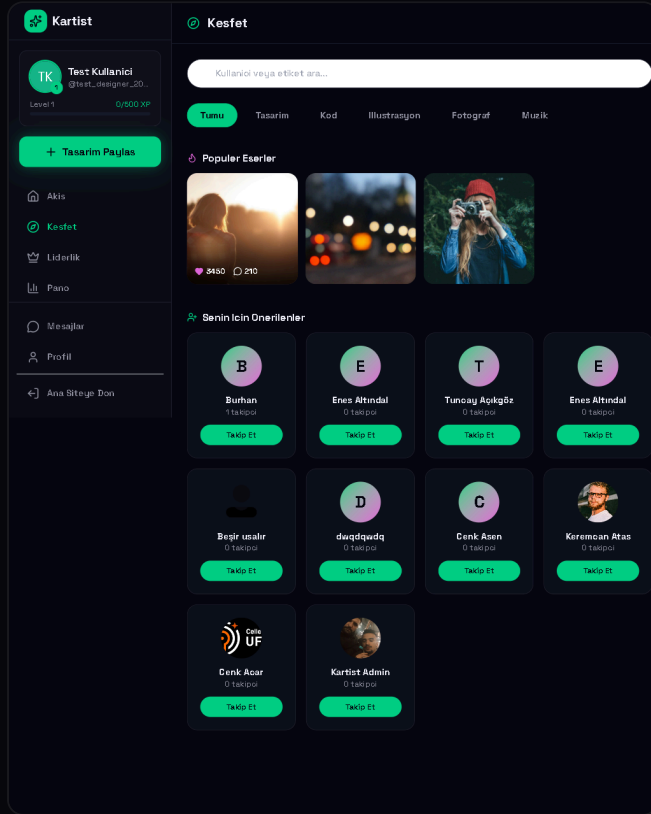
🚩 **Vize Kilometre Taşı:** 29 Mart'ta MVP canlıya alındı ve vize raporu teslim edildi. Bu noktadan sonra proje, sosyal ağ özellikleriyle genişlemeye başladı.

HAFTA 5-6 31 Mart-13 Nisan Feed, gönderi CRUD, beğeni, yorum, takip, hikâyeler, AI moderasyon

Tasarımcı Topluluğu

✓ TAMAMLANDI

- ✓ **SOSYAL AKIŞ** Gönderi oluşturma/düzenleme/silme, beğeni (SosyalBegeniler), iç içe yorum (UstYorumId ile yanıt), takip (Takipciler).
- ✓ **HİKÂYELER** 24 saatlik Hikayeler kayıtları ve keşfet (Kesk) sayfası.
- ✓ **AI MODERASYON** Tüm kullanıcı metni (AiModerationService) gönderi/yorum oluşturma akışında zararlı içerik için süzülür.
- ✓ **XSS GÜVENCESİ** Tüm UGC, innerHTML öncesi escapeHtml ile kaçışlanır; depolama öncesi SanitizeHtml.



Şekil 4 — Keşfet sayfası ve kullanıcı profili (takipçi, tasarımlar, rozetler)

```
// SocialService - gönderi oluşturmadan önce AI moderasyon
var mod = await _moderation.CheckAsync(icerik);
if (!mod.IsSafe) return Fail(mod.Reason);
icerik = InputValidator.SanitizeHtml(icerik); // script/iframe/on* temizle
await _repo.CreatePostAsync(userId, icerik, mediaUrl);
```

HAFTA 7 14–20 Nisan Direkt mesaj (DM), Instagram tarzı arayüz, kullanıcı keşfi**HAFTA 8** 21–27 Nisan Sprint 5: Güvenlik sertleştirme (HMAC deploy, dosya doğrulama, çerez)**Üretim Sınıfı Güvenlik**

✓ TAMAMLANDI

- ✓ **MESAJLAŞMA** DirektMesajlar tablosu; Instagram/WhatsApp tarzı sohbet arayüzü; site geneli kullanıcı keşfi.
- ✓ **HMAC DEPLOY** Deploy endpoint'indeki sabit parola kaldırıldı; **HMAC-SHA256 imzalı** ve zaman damgalı kimlik doğrulamaya geçildi.
- ✓ **DOSYA DOĞRULAMA** Yüklenen görseller **magic-byte** ile doğrulanır (JPEG/PNG/GIF/WEBP); dosya adı/uzantı istemciden ASLA türetilmez.
- ✓ **ADMIN YETKİ** Tüm admin mutasyonları [HttpPost]+[ValidateAntiForgeryToken] ve fail-closed yetki kontrolü.



Şekil 5 — Direkt Mesajlaşma (DM) arayüzü

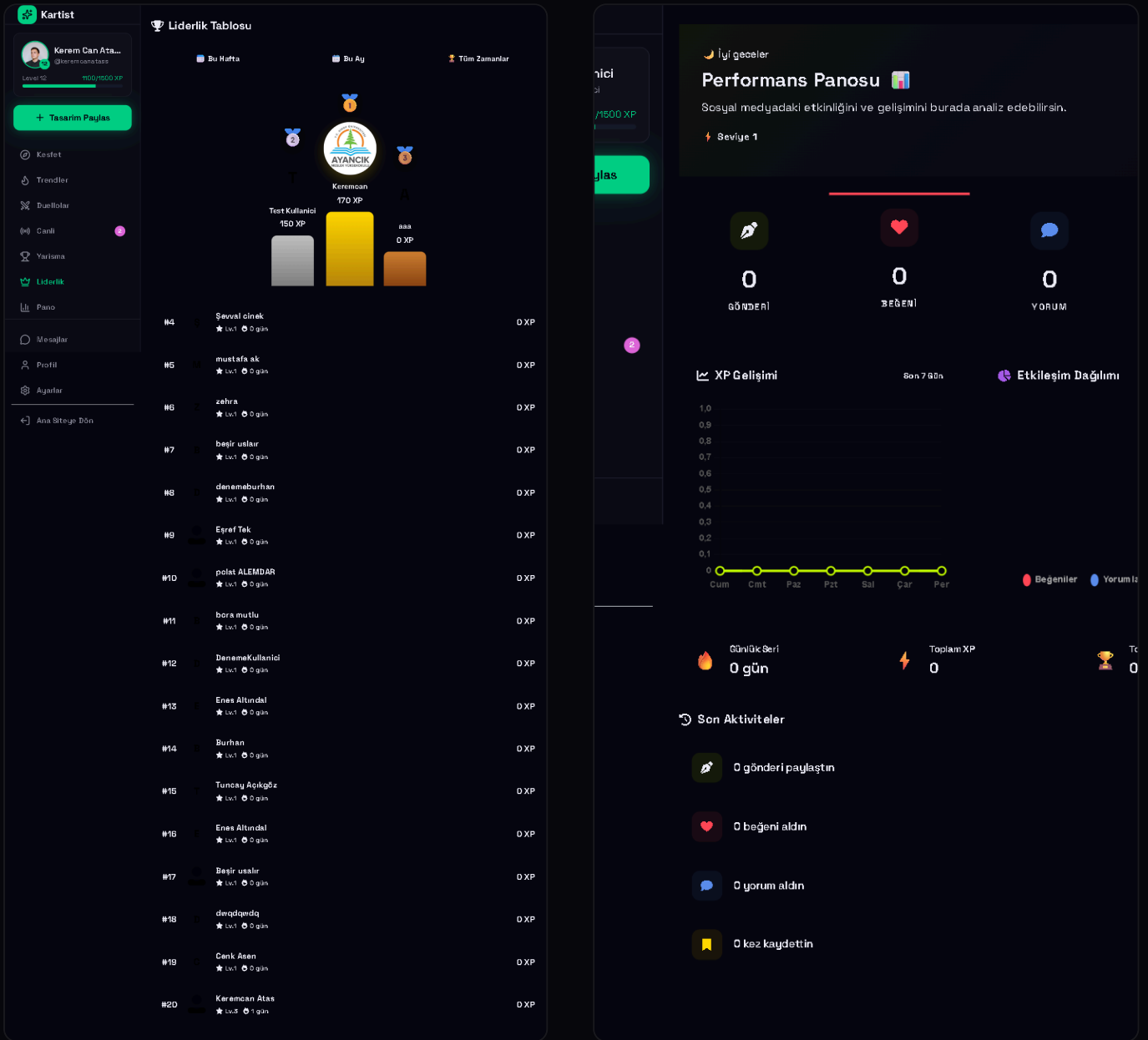
```
// HMAC-SHA256 imzalı deploy doğrulaması (Program.cs)
using var hmac = new HMACSHA256(Encoding.UTF8.GetBytes(secret));
var hash = hmac.ComputeHash(Encoding.UTF8.GetBytes(timestamp));
var expected = Convert.ToHexString(hash).ToLowerInvariant();
// Zamanlama saldırısına dayanıklı sabit-sürekli karşılaştırma
return CryptographicOperations.FixedTimeEquals(provided, expected);
```

HAFTA 9 28 Nisan–4 Mayıs XP & seviye, rozetler, günlük görevler, liderlik tablosu, istatistik

Etkileşim Ekonomisi

✓ TAMAMLANDI

- ✓ **XP & SEVİYE** KullanıcıXP işlem defteri ana kaynaktır; ToplamXP ve Seviye önbelleklenmiş alanlardır.
- ✓ **ROZETLER** Rozetler + KullanıcıRozetleri; başarımlar otomatik tetiklenir.
- ✓ **GÜNLÜK GÖREVLER** GunlukGorevler ile günlük ilerleme ve ödül takibi.
- ✓ **LİDERLİK** ToplamXP DESC sıralı liderlik tablosu ve istatistik panosu.



Şekil 6 — Liderlik tablosu ve kullanıcı istatistik panosu

 **Veri Modeli Notu:** XP "tek doğruluk kaynağı" prensibiyle bir işlem defterinde (ledger) tutulur; toplam ve seviye performans için önbelleklenir. Bu, finansal sistemlerdeki muhasebe defteri yaklaşımının küçük ölçekli bir uygulamasıdır.

HAFTA 10 5–11 Mayıs WebRTC P2P canlı yayın, SignalR sinyalizasyon, canlı sohbet, düello

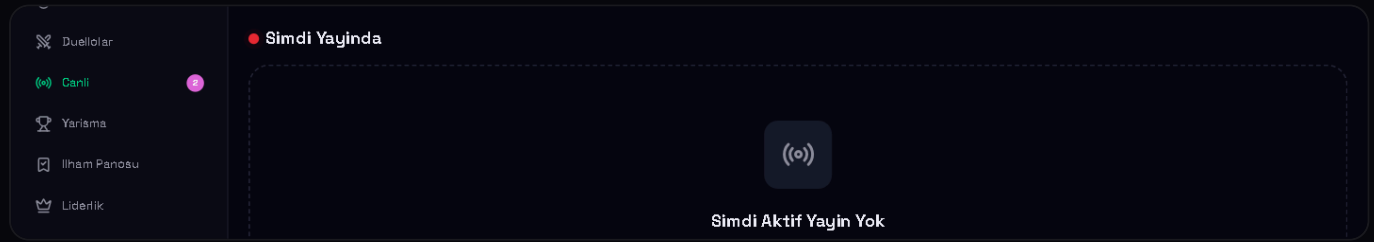
Dersin Kalbi: Canlı yayın özelliği, "İnternet ve Ağ Teknolojileri" dersinin temel konularını tek bir özellikte birleştirir: **WebSocket (SignalR)** ile sinyalizasyon, **WebRTC** ile uçtan uca (P2P) medya akışı, **STUN/ICE** ile NAT arkasındaki cihazların birbirini bulması.

Sinyalizasyon & Bağlantı Akışı

```
// 1) Yayınçı → SignalR hub'a yayını kaydeder
Yayınçı —StartBroadcast→ NotificationHub (WebSocket)
// 2) İzleyici katılır, hub yayıncıya haber verir
İzleyici —JoinStream→ Hub —ViewerJoined→ Yayınçı
// 3) SDP teklif/yanıt + ICE adayları hub üzerinden taşınır
Yayınçı ⇄ SendWebRTCoffer/Answer ⇄ SendICECandidate ⇄ İzleyici
// 4) STUN ile yol bulunur → medya artık DOĞRUDAN P2P akar
Yayınçı ===== WebRTC P2P (medya sunucusu YOK) ===== İzleyici
```

```
// STUN sunucuları ile ICE — yerel ağda DOĞRUDAN P2P bağlantı
var rtcConfig = {
  iceServers: [
    { urls: 'stun:stun.l.google.com:19302' },
    { urls: 'stun:stun1.l.google.com:19302' }
  ]
};

// Hub tarafı sinyalizasyon (C#)
public async Task SendICECandidate(string target, string cand)
⇒ await Clients.Client(target).SendAsync("ReceiveICECandidate",
Context.ConnectionId, cand);
```



Şekil 7 — Canlı Yayın sayfası: aktif yayın listesi, izleyici sayacı ve canlı sohbet

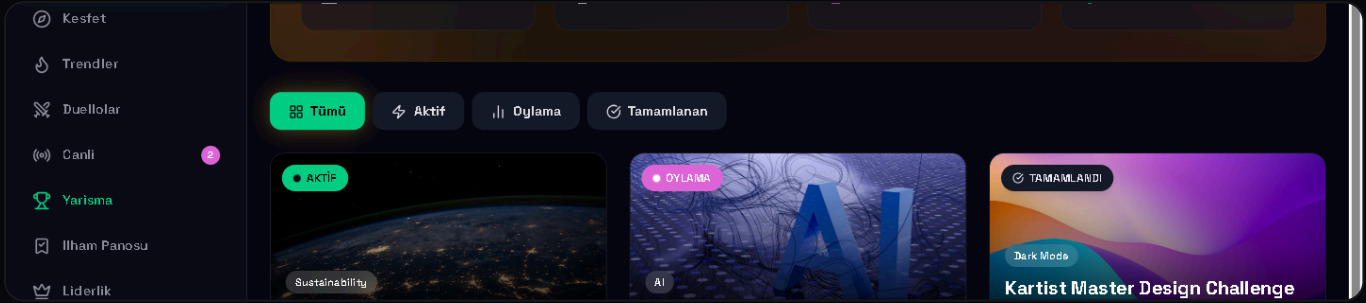
HAFTA 11 12-18 Mayıs SMTP/deploy kararlılığı, AI yazar sağlayıcı fallback'ı

HAFTA 12 19-29 Mayıs AI jürili yarışmalar, 3 varyasyon AI Studio, kurumsal harita, canlı yayın optimizasyonu

AI Jürili Tasarım Yarışmaları

✓ TAMAMLANDI

- ✓ **AI JÜRİ** AiJuryService, gönderilen tasarımları kriterlere göre puanlar; otomatik değerlendirme.
- ✓ **YAŞAM DÖNGÜSÜ** YarismaLifecycleService (arka plan servisi) yarışma durumunu (aktif→oylama→bitti) ve kazananı yönetir.
- ✓ **KULLANICI AKIŞI** Stüdyodan tasarla → katıl → oyla → sonuçları gör; gerçek katılım/oy istatistikleri.
- ✓ **YARIŞMALAR SAYFASI** Final öncesi tamamen yeniden tasarlandı: mantık hatasız, her statüye tek tutarlı eylem.



Şekil 8 — Yeniden tasarlanan Yarışmalar sayfası: gerçek istatistikler ve durum bazlı eylem butonları

Final Cılası: 3 varyasyonlu AI Studio galerisi, Türkçe→İngilizce sadık görsel prompt çevirisi, kurumsal sayfada WebGL gerektirmeyen canlı OSM/Leaflet harita ve canlı yayında yankı/kasma optimizasyonu (echo cancellation + bitrate/çözünürlük tavanı) tamamlandı.

```
Kartist/ ├── .github/workflows/ └─ dotnet.yml ← CI + imzalı deploy ├── Controllers/ │ ├── Base/
BaseController.cs ← CurrentUserId/Email │ ├── SocialController.cs ← feed, canlı, yarışma │ ├──
HomeController.cs · AccountController.cs · AdminController.cs ├── Hubs/ └─ NotificationHub.cs ← SignalR +
WebRTC sinyal ├── Services/ │ ├── AiPromptService.cs · AiImageService.cs ← LLM + görsel │ ├──
AiModerationService.cs ← UGC moderasyon │ ├── Business/ SocialService.cs │ ├── AiJuryService.cs ·
YarismaLifecycleService.cs ├── Data/ │ ├── DatabaseInitializer.cs ← idempotent şema │ └─ Repositories/
SocialRepository.cs ├── Helpers/ InputValidator.cs · FileUploadValidator.cs ├── Middleware/ SecurityHeaders ·
RateLimiting ├── Views/ │ ├── Home/ Tasarim.cshtml ← AI editör tuvali │ └─ Social/ Feed · Live · Competitions
· Duels · Messages ├── wwwroot/ └─ js/kartist-social.js ← CSRF, escapeHtml, SignalR ├── Program.cs ← boru
hattı + /api/deploy └─ Kartist.csproj ← net8.0
```

5

CONTROLLER

6+

SERVİS

1

SİGNALR HUB

21+

DB TABLOSU

3

AI SERVİSİ

12

HAFTA

9+

GÜVENLİK KATMANI

v3.0

SÜRÜM

Türkçe Şema (seçki): Kullanıcılar, Yöneticiler, Sablonlar, SosyalGonderiler, SosyalBegeniler, SosyalYorumlar, Takipciler, Hikayeler, DirektMesajlar, Bildirimler, Rozetler, KullanıcıXP, GunlukGorevler, IkiFactorKodlari, GirisLoglari, CanlıYayınlar, Yarismalar, YarismaKatilimlari, YarismaOylari.

12 haftalık iteratif geliştirme sonunda KARTIST, vize tesliminde planlanan tüm hedefleri aşmış; bir tasarım editöründen **AI destekli, gerçek zamanlı, oyunlaştırılmış bir tasarımcı sosyal ağına** dönüşmüştür. Aşağıdaki dört sütun, dersin kazanımlarının projeye nasıl yansıdığını özetler:



Ağ Teknolojileri ✓

HTTP/HTTPS REST, SignalR (WebSocket), WebRTC P2P, STUN/ICE NAT geçişi, TLS, OAuth 2.0

Güvenlik ✓

BCrypt, 2FA, CSRF, CSP, rate limit, magic-byte yükleme, HMAC-SHA256 imzalı deploy



Yapay Zekâ ✓

Gemini/Groq LLM, AI görsel üretimi + fallback, içerik moderasyonu, yarışma jürisi



DevOps ✓

GitHub Actions CI, IIS/Plesk, app_offline.htm ile sıfır-veri-kaybı deploy, canlı demo

🚩 Final Değerlendirme Beyanı:

Keremcan ATAŞ (247017033) tarafından geliştirilen **KARTIST v3.0** projesi; ASP.NET Core 8.0, Dapper, SignalR + WebRTC gerçek zamanlı altyapısı, çok sağlayıcılı yapay zekâ entegrasyonu ve endüstri standartlarında güvenlik katmanlarıyla **dönem hedeflerinin tamamına ulaşmıştır**.

Platform **kartistt.com.tr** adresinde canlı olarak hizmet vermektedir. Test erişimi ve kullanım adımları için ekteki **"Kartist Kullanım Rehberi"** dokümanına bakınız.